Trading Strategy Lab

Factual product brief for the current application. Component, schema, and dependency detail lives in the companion documents in this folder. Operator procedures live in backend/README.md.

Source of truth: application code, Alembic revisions, and `.env.example`. This brief does not add behaviour that the code does not implement.

Analysis date: 2026-10-08.

Problem statement

A retail options idea usually arrives as a video, a caption, a note, or a Markdown write-up. Turning that into a strategy that can be backtested with Indian options charges, reviewed trade by trade, and only then approved for paper or live trading is a manual process. The same idea is easy to change between the backtest and the orders that actually go out.

The lab ingests a strategy, runs a Review Crew that backtests and forward-tests it, and keeps an append-only record of every version, report, trade, and review. A later approval freezes one exact version. A Trade Crew may then trade that frozen copy on paper or, when the live gates pass, through a broker adapter. An LLM extracts and comments. It does not open or close a trade.

Current situation

1. A strategy is pasted, fetched from YouTube, Instagram, or an HTML page, or loaded from the demo fixture.

2. An extractor turns it into Markdown with a `rules:` block. Anything it cannot express is listed under Gaps.

3. The Review Crew replays the rules on imported NSE end-of-day data, or on the bundled synthetic sample when that data is missing, and writes a verdict, trades, reviews, and improvement proposals.

4. A person approves one version for a broker account. Approval copies that version into the live database.

5. A separate Trade Crew process ticks that deployment. It checks the frozen rules in code, then sends orders through the account's broker adapter.

The cost of the problem

- Time spent rewriting a video into rules that a backtest can run.
- Backtests that ignore brokerage, STT, exchange fees, GST, SEBI charges, stamp duty, and slippage.
- A live order that does not match the version that was reviewed.
- No record of why a trade was taken, what it cost, or which review corrected an earlier one.

Why a notes-only workflow fails

- The write-up and the executable rules can diverge.
- A verdict from fixed scenarios is not the same test as the strategy's own `rules:` block.
- Paper and live fills, margin, and protective stops are not visible in a spreadsheet of hypothetical trades.
- Without an append-only log, a later edit can erase the version that was approved.

Project goal

Provide one path from an ingested strategy to a reviewed report and, only after an explicit approval, to a Trade Crew that trades that frozen version.

The system must:

- Store every strategy version, report, simulated trade, review, and live row so it can be audited.
- Apply an Indian options cost model.
- Refuse live trading unless the configured gates pass.
- Keep LLM output out of the order path except for an explicit, evidenced veto that can only skip or shrink an entry the rules already approved.

Core requirements

Ingest and extract

Accept `youtube`, `instagram`, `document` (HTML), `notebooklm`, or `markdown`. Persist the extractor name, model, and raw output on `ingest_runs`. PDF URLs are rejected.

Review Crew

A LangGraph workflow audits the strategy, builds scenarios, simulates backtest and forward windows, sweeps hard stops, writes a review per simulated trade, synthesizes a verdict, and proposes improvements. The verdict label comes from the scenarios. It does not come from the hard-stop sweep.

Approval and Trade Crew

Approval freezes `content_sha256` and `rules_sha256`. The Trade Crew halts if those hashes change. Each tick verifies the snapshot, checks the deployment is started, observes the market, reconciles positions, exits, applies drawdown and the capital floor, then considers an entry. Orders go through one order lifecycle for paper and live.

Human control

Archive is a soft delete. Reviews can supersede earlier reviews. Improvement decisions are accepted, rejected, or applied. Applied requires new Markdown and creates a new strategy version. Stop is final. Flatten is a separate, token-gated kill switch.

Success criteria that the code actually enforces

These are checks in the application, not measured service-level objectives. There is no latency SLO and no RAGAS score in this repository.

- A report stores cost profile, settings, a market-data fingerprint (source, date range, and a sha256 of imported file hashes), engine version, review-rules version, git commit, and app version.
- Research and live audit logs are hash chains. `GET /api/audit/verify` and `GET /api/live/audit/verify` recompute them.
- Amounts use `Numeric(18, 4)`.
- Live approval requires parsed rules, empty Gaps, a matching `confirm_rules_sha256`, a completed report, `X-Actor` or a token, a mode-matching account, and no other active deployment of that strategy in that mode.
- Live mode also requires `LIVE_TRADING_ENABLED`, a live-capable registered broker, resolved credentials, an allowed structure, and `LIVE_MIN_PAPER_DAYS` when that setting is above zero.
- Tests cover the research schema, the live schema, the Trade Crew, and the Sandbox broker. No test calls a real LLM or a real broker.

Technical scope

1. API (FastAPI).

2. Review Crew and Trade Crew (LangGraph).

3. Research database, live database, and an unaudited market database.

4. Broker adapters (Paper and Sandbox). No Zerodha, Upstox, Dhan, or Angel One adapter ships in this tree.

5. Audit hash chains, heartbeats, and Telegram or SMTP alerts.

6. Next.js UI for the lab, deploy, and deployments.

`vendor/TradingAgents` is an architecture reference. Runtime code does not import it. See vendor/INTEGRATION.md.

Sample data

- `backend/fixtures/notebooklm_notes.txt` is the demo strategy source. It omits `short_distance`, so it cannot be approved.
- Without imported NSE data, simulations use the bundled synthetic sample and the report says so (`data_source`).
- Holidays are listed in `backend/data/holidays/nse.yaml`. Events are YAML under `backend/data/events/` plus an optional NSE board-meetings file.

High-risk cases the code is written to catch

- Gaps still present, or `confirm_rules_sha256` not matching the preview.
- A live approval while `LIVE_TRADING_ENABLED` is false, credentials are missing, or the structure is not on `LIVE_ALLOWED_STRUCTURES`.
- A snapshot hash mismatch, a broker position that differs from the live database, or a tick that raises.
- A failed entry leg (filled legs are unwound, then the deployment halts).
- A protective stop that cannot be placed (the entry is unwound and the deployment halts).
- A kill switch whose cancel or flatten fails (the deployment stays stopped and an alert is sent).

Documents in this folder

Document	Role
CURRENT_SYSTEM_ARCHITECTURE_ANALYSIS.md	Components, flows, and diagram input from the current code
CURRENT_DATABASE_ARCHITECTURE.md	Schema of the three databases
DATABASE_TABLE_REFERENCE.md	What each table is for and who writes it
LIBRARIES_AND_DEPENDENCIES.md	Libraries that the code actually imports
FINAL_REPORT.md	Report in the same section order as the NIIT final report

Word copies are in docx/. PDF copies are in pdf/. Diagrams are in diagrams/. Regenerate them with `python3 DocumentsAndDiagrams/export_docs.py`.

Getting started

Setup, ports, and the demo command are in the repository README.md. Database migrations, the API, and the live runbook are in backend/README.md.

Important note

Report verdicts come from scenario windows. They are not evidence that the strategy's own `rules:` block made money. The approval backtest (`POST /api/live/approvals/backtest`) is the rules replay. Live trading still needs a registered live-capable broker adapter; Paper and Sandbox are the adapters in this repository. Sandbox live mode exists only when `SANDBOX_ALLOW_LIVE=true`, and it does not send orders to an exchange.